

GLSL Shader Examples Without External Libraries

This document contains:

1. A simple orthographic GLSL shader setup
2. A simple perspective GLSL shader setup
3. Manual matrix math implementation in C++
4. No external libraries such as GLM

Everything is implemented manually.

Basic Matrix Implementation

We first create a minimal 4x4 matrix system.

Matrix4.h

```
#pragma once

#include <cmath>

struct Mat4
{
    float m[16];

    static Mat4 Identity()
    {
        Mat4 result = {};

        result.m[0] = 1.0f;
        result.m[5] = 1.0f;
        result.m[10] = 1.0f;
        result.m[15] = 1.0f;

        return result;
    }

    static Mat4 Orthographic(
        float left,
        float right,
        float bottom,
        float top,
        float nearPlane,
```

```

        float farPlane)
    {
        Mat4 result = Identity();

        result.m[0] = 2.0f / (right - left);
        result.m[5] = 2.0f / (top - bottom);
        result.m[10] = -2.0f / (farPlane - nearPlane);

        result.m[12] = -(right + left) / (right - left);
        result.m[13] = -(top + bottom) / (top - bottom);
        result.m[14] = -(farPlane + nearPlane) / (farPlane - nearPlane);

        return result;
    }

    static Mat4 Perspective(
        float fov,
        float aspect,
        float nearPlane,
        float farPlane)
    {
        Mat4 result = {};

        float tanHalfFov = tanf(fov * 0.5f);

        result.m[0] = 1.0f / (aspect * tanHalfFov);
        result.m[5] = 1.0f / tanHalfFov;

        result.m[10] = -(farPlane + nearPlane) /
            (farPlane - nearPlane);

        result.m[11] = -1.0f;

        result.m[14] = -(2.0f * farPlane * nearPlane) /
            (farPlane - nearPlane);

        return result;
    }

    static Mat4 Translation(float x, float y, float z)
    {
        Mat4 result = Identity();

        result.m[12] = x;
        result.m[13] = y;
        result.m[14] = z;

        return result;
    }
};

```

1. Orthographic Shader

Orthographic rendering keeps object size constant regardless of distance.

orthographic.vert

```
#version 330 core

layout(location = 0) in vec3 aPosition;
layout(location = 1) in vec3 aColor;

out vec3 vColor;

uniform mat4 uModel;
uniform mat4 uView;
uniform mat4 uProjection;

void main()
{
    vColor = aColor;

    gl_Position =
        uProjection *
        uView *
        uModel *
        vec4(aPosition, 1.0);
}
```

orthographic.frag

```
#version 330 core

in vec3 vColor;

out vec4 FragColor;

void main()
{
    FragColor = vec4(vColor, 1.0);
}
```

Orthographic Projection Setup

```
Mat4 projection = Mat4::Orthographic(  
    -10.0f,  
    10.0f,  
    -10.0f,  
    10.0f,  
    0.1f,  
    100.0f  
);
```

2. Perspective Shader

Perspective rendering simulates realistic depth.

perspective.vert

```
#version 330 core  
  
layout(location = 0) in vec3 aPosition;  
layout(location = 1) in vec3 aColor;  
  
out vec3 vColor;  
  
uniform mat4 uModel;  
uniform mat4 uView;  
uniform mat4 uProjection;  
  
void main()  
{  
    vColor = aColor;  
  
    gl_Position =  
        uProjection *  
        uView *  
        uModel *  
        vec4(aPosition, 1.0);  
}
```

perspective.frag

```
#version 330 core

in vec3 vColor;

out vec4 FragColor;

void main()
{
    FragColor = vec4(vColor, 1.0);
}
```

Perspective Projection Setup

```
float fov = 45.0f * 3.14159265f / 180.0f;
float aspect = 1280.0f / 720.0f;

Mat4 projection = Mat4::Perspective(
    fov,
    aspect,
    0.1f,
    100.0f
);
```

Basic View Matrix

A very simple camera translation.

Simple Camera View Matrix

```
Mat4 view = Mat4::Translation(0.0f, 0.0f, -5.0f);
```

This moves the world backward by 5 units.

Uploading Matrices To OpenGL

```
glUniformMatrix4fv(
    projectionLocation,
```

```
    1,  
    GL_FALSE,  
    projection.m  
);  
  
glUniformMatrix4fv(  
    viewLocation,  
    1,  
    GL_FALSE,  
    view.m  
);  
  
glUniformMatrix4fv(  
    modelLocation,  
    1,  
    GL_FALSE,  
    model.m  
);
```

Difference Between Orthographic And Perspective

Orthographic

Objects keep the same size regardless of depth.

Useful for:

- 2D games
- UI rendering
- tilemaps
- CAD tools
- strategy games

Perspective

Far objects appear smaller.

Useful for:

- FPS games
- third-person games
- simulation

- realistic 3D rendering
-

Matrix Layout Notes

This implementation uses:

Column-major layout

Compatible with OpenGL expectations.

Recommended File Layout

```
/src
  Matrix4.h

/shaders
  orthographic.vert
  orthographic.frag
  perspective.vert
  perspective.frag
```

Minimal Rendering Pipeline

```
Vertex Data
  ↓
Model Matrix
  ↓
View Matrix
  ↓
Projection Matrix
  ↓
Vertex Shader
  ↓
Clip Space
  ↓
Rasterization
  ↓
Fragment Shader
```

Summary

This implementation:

- Uses no external math libraries
- Implements orthographic projection manually
- Implements perspective projection manually
- Uses raw OpenGL-compatible matrices
- Demonstrates the complete projection pipeline
- Is suitable for small engines and learning graphics programming